```
root@ubuntu:~$ docker run -d -p 8080:8080 --name airflow apache/airflow standalone
Unable to find image 'apache/airflow:latest' locally
latest: Pulling from apache/airflow
d96c629aea05: Pull complete
7dd375afdfa9: Pull complete
2a3a3b117b57: Pull complete
122b3d8130d6: Pull complete
14dec0eb9397: Pull complete
4f4fb700ef54: Pull complete
c1b5a06a80a7: Pull complete
ec476eeda31b: Pull complete
d8e30ea1698a: Pull complete
f993bee01a0c: Pull complete
b19a6b881c51: Pull complete
f84e5ac072fa: Pull complete
27f90811415c: Pull complete
358d4451bcf8: Pull complete
c559cfd41af0: Pull complete
62ee21ebdf5c: Pull complete
4b3838502dbc: Pull complete
8809629c72a2: Pull complete
c9492c537c93: Pull complete
6b9a7beea5b3: Pull complete
46d0fd3752bd: Pull complete
a375fb8a9b8b: Pull complete
93151ca22bef: Pull complete
Digest: sha256:a8d9d19c563c768b2bc605ef26440f28a9164ea886064e99c599b057432ff572
Status: Downloaded newer image for apache/airflow:latest
fb8df4213e7ad4a5c888712d959d1b52d2caf15272be2d18bf2452554237d081
root@ubuntu:~$ docker ps
```

CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES

fb8df4213e7a apache/airflow "/usr/bin/dumb-init …" 43 seconds ago Up 42 seconds 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp airflow

root@ubuntu:~$ docker logs airflow

standalone | Starting Airflow Standalone

standalone | Checking for new Dag files every 5 seconds (set [dag_processor] refresh_interval to change this)

2026-11-12T11:03:39.758398Z [warning] SimpleAuthManager is active but the deployment shape looks like production (non-sqlite backend, non-local API host, or a distributed executor). SimpleAuthManager stores passwords in plaintext at /opt/airflow/simple_auth_manager_passwords.json.generated and prints generated passwords to stdout/logs on first init. Use a real auth manager (e.g. FAB or Keycloak) for production deployments. [airflow.api_fastapi.auth.managers.simple.simple_auth_manager] loc=simple_auth_manager.py:179

Simple auth manager | Password for user 'admin': wMbAhMgSD69mZEEX

standalone | Checking database is initialized

2026-11-12T11:03:40.870735Z [info] Creating Airflow database tables from the ORM [airflow.utils.db] loc=db.py:750

2026-11-12T11:03:40.874850Z [info] Creating global lock context [airflow.utils.db] loc=db.py:731

2026-11-12T11:03:40.875014Z [info] Pool status: Pool size: 5 Connections in pool: 0 Current Overflow: -4 Current Checked out connections: 1 [airflow.utils.db] loc=db.py:734

2026-11-12T11:03:40.875080Z [info] Creating metadata [airflow.utils.db] loc=db.py:736

2026-11-12T11:03:41.110532Z [info] Getting alembic config [airflow.utils.db] loc=db.py:739

2026-11-12T11:03:41.116837Z [info] Stamping migration head [airflow.utils.db] loc=db.py:742

2026-11-12T11:03:41.281246Z [info] Airflow database tables created [airflow.utils.db] loc=db.py:745

2026-11-12T11:03:41.282610Z [info] Creating FABDBManager tables from the ORM [airflow.providers.fab.auth_manager.models.db.FABDBManager] loc=db_manager.py:137

2026-11-12T11:03:41.328570Z [info] FABDBManager tables have been created from the ORM [airflow.providers.fab.auth_manager.models.db.FABDBManager] loc=db_manager.py:143

standalone | Database ready

dag-processor | 2026-11-12T11:03:53.105639Z [info] Found 0 files for bundle dags-folder (dag_discovery_safe_mode=True) [airflow.dag_processing.manager.DagFileProcessorManager] loc=manager.py:1033

dag-processor | 2026-11-12T11:03:53.180355Z [info]

dag-processor |
==========

dag-processor | DAG File Processing Stats

dag-processor | Bundle File Path PID Current Duration # DAGs # Errors Last Duration Last Run At

dag-processor | -------- ----------- ----- ------------------ -------- ---------- --------------- -------------

dag-processor | ==========
[airflow.dag_processing.manager.DagFileProcessorManager] loc=manager.py:1224

scheduler | 2026-11-12T11:03:53.656220Z [info] Worker starting up pid=53 [airflow.executors.local_executor.LocalExecutor] loc=local_executor.py:83

root@ubuntu:~$ docker exec -it airflow bash

airflow@fb8df4213e7a:/opt/airflow$ cd /opt/airflow/dags

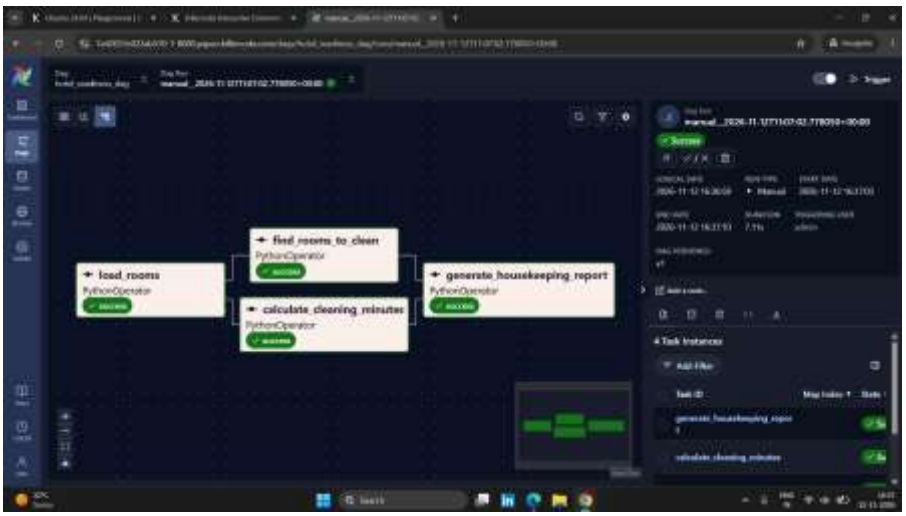
airflow@fb8df4213e7a:/opt/airflow/dags$ ls

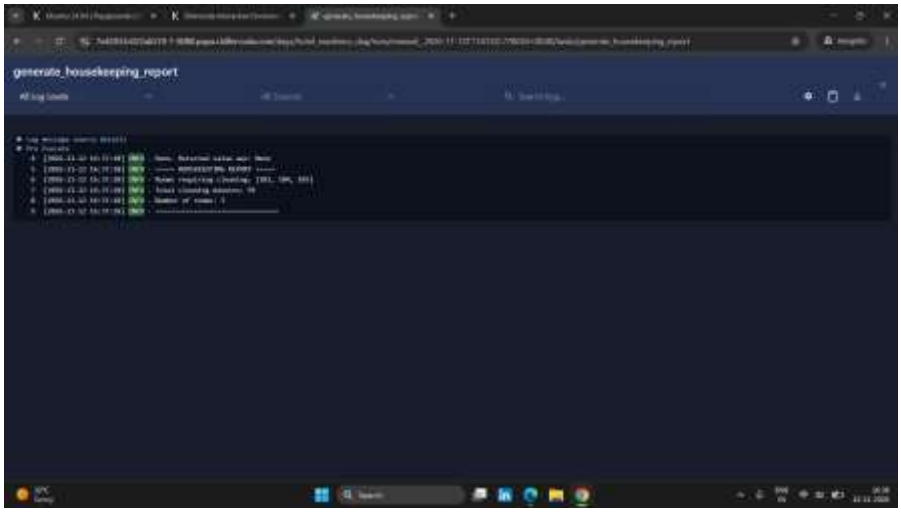
airflow@fb8df4213e7a:/opt/airflow/dags$ cat > hotel_readiness_dag.py <<'EOF'

>

```
from datetime import datetime
from airflow import DAG
from airflow.providers.standard.operators.python import PythonOperator
# Task 1: Load room details
def load_rooms():
    rooms = [
        {"room": 101, "status": "Ready", "cleaning_min": 0},
        {"room": 102, "status": "Cleaning", "cleaning_min": 25},
        {"room": 103, "status": "Ready", "cleaning_min": 0},
        {"room": 104, "status": "Cleaning", "cleaning_min": 40},
        {"room": 105, "status": "Cleaning", "cleaning_min": 30}
    ]
    return rooms
# Task 2: Find rooms that need cleaning
def find_rooms_to_clean(ti):
    rooms = ti.xcom_pull(task_ids="load_rooms")
```

```
    cleaning_rooms = [
        room["room"] for room in rooms
        if room["status"] == "Cleaning"
    ]
    print("Rooms to clean:", cleaning_rooms)
    return cleaning_rooms
# Task 3: Calculate total cleaning time
def calculate_cleaning_minutes(ti):
    rooms = ti.xcom_pull(task_ids="load_rooms")
    [find_cleaning, calculate_time] >> report_reports)
> EOF
airflow@fb8df4213e7a:/opt/airflow/dags$ python -m py_compile hotel_readiness_dag.py
airflow@fb8df4213e7a:/opt/airflow/dags$ airflow dags list
```





```
airflow@fb8df4213e7a:/opt/airflow/dags$ cd /opt/airflow/dags
airflow@fb8df4213e7a:/opt/airflow/dags$ cat > it_incident_routing_dag.py <<'EOF'
>
from datetime import datetime, timedelta

from airflow import DAG
from airflow.providers.standard.operators.python import (
    PythonOperator,
    BranchPythonOperator
)
from airflow.utils.trigger_rule import TriggerRule



# Task 1: Load incident details
def load_incident():
    incident = {
        "ticket_id": "INC-402",
        "service": "Email",
        "severity": "High",
        "reported_by": "Operations"
    }
    return incident
# Task 2: Choose response based on severity
def choose_response(ti):
    incident = ti.xcom_pull(task_ids="load_incident")
    severity = incident["severity"]
    if severity in ["High", "Critical"]:
        print("High priority incident detected!")
        return "urgent_response"
    else:
        print("Normal priority incident detected!")
```

```
    return "standard_response"

# Task 3: Handle urgent incidents

  [urgent, standard] >> audit].NONE_FAILED_MIN_ONE_SUCCESS")

> EOF

airflow@fb8df4213e7a:/opt/airflow/dags$ python -m py_compile it_incident_routing_dag.py

airflow@fb8df4213e7a:/opt/airflow/dags$ airflow dags list


airflow@fb8df4213e7a:/opt/airflow/dags$ sed -i 's/"severity": "High"/"severity": "Low"/'
it_incident_routing_dag.py

airflow@fb8df4213e7a:/opt/airflow/dags$ python -m py_compile it_incident_routing_dag.py
```

